

LC 2818: Apply Ops to Maximize Score

Authored by: Arpit Raj, LNMIIT Jaipur | September 26, 2026

CORE OPERATIONS & RULES

- At max k operations can be performed. Start with a score of 1.
- In one op:** Choose any subarray $\rightarrow$ choose element x with the highest prime score.
- [Prime score = number of distinct prime factors]*
- If there are multiple x , choose the one with the smallest index.
- Multiply the total score by x .

EXAMPLES

Example 1

Array: [8, 3, 9, 3, 8] | $k = 2$

Each of these numbers has a prime score of 1.

Valid operations:

- Subarray [2, 2] = 9
- Subarray [2, 3] = 9

Result: $1 \times 9 \times 9 = 81$

Example 2

Array: [19, 12, 14, 6, 10, 18] | $k = 3$

Selected maximums across valid subarrays:

Result: $19 \times 18 \times 14 = 4788$

Important Observation

We can't keep selecting 19, 19, 19 to make $19 \times 19 \times 19$. One subarray can be selected **only once**.

.

Hence:

$\text{left}[i] \rightarrow$ no of valid start positions.

$\text{right}[i] \rightarrow$ no of valid end positions.

$\text{ways} = \text{left}[i] * \text{right}[i] \rightarrow \text{nums}[i]$ can be chosen in these many subarrays.

VALID STARTS AND ENDS LOGIC

LEFT = valid starts

Let $\text{currnum} \rightarrow$ at i index.

Then move left while $\text{score}[j] < \text{score}[i]$.

Stop if $\text{score}[j] \geq \text{score}[i]$ cause then i index won't remain biggest of house.

RIGHT = valid ends

Simply move right till you spot a strictly greater score ($\text{score}[j] > \text{score}[i]$).

APPROACH 1: $O(N^2)$ LEFT/RIGHT BOUNDS

Initial brute-force approach to find the left and right bounds using while loops. (Expected to TLE)

Code Iteration 1 - Part 1

```
class Solution {
public:
    int maximumScore(vector<int>& nums, int k) {

        const long long MOD=1e9+7;
        int n=nums.size();
        vector<int> score(n,0);
        for(int i=0;i<n;i++){

            int temp=nums[i];
            for(int p=2;p*p<=temp;p++){
                if(temp%p==0){
                    score[i]++;
                    while(temp%p==0){
                        temp/=p;
                    }
                }
            }

            if(temp>1){
                score[i]++;
            }
        }

        vector<long long> ways(n);
        for(int i=0;i<n;i++){

            int l=i;
            while(l>=1 && score[l-1]<score[i]){
                l--;
            }

            int r=i;
            while(r<n-1 && score[r+1]<=score[i]){
                r++;
            }

            ways[i]=1LL*(i-l+1)*(r-i+1);
        }
    }
};
```

Code Iteration 1 - Part 2

```
vector<pair<int,int>> arr;
for(int i=0;i<n;i++){
    arr.push_back({nums[i],i});
}

sort(arr.begin(),arr.end(),greater<pair<int,int>>());

long long ans=1;
for(auto it:arr){

    if(k==0) break;
    int val=it.first;
    int idx=it.second;

    long long take=min(ways[idx],(long long)k);
    long long curr=1;

    while(take>0){
        curr=curr*val%MOD;
        take--;
    }

    ans=ans*curr%MOD;
    k-=min(ways[idx],(long long)k);
}

return ans;
};
```

APPROACH 2: RESOLVING TLE WITH BINARY EXPONENTIATION

Explanation Language Note:

"This part was leading to a TLE, so we use binary exponentiation.

Aaj ka gyaan, mujhe bhi aaj hi pata chala."

Code Iteration 2 - Part 1

```
class Solution {
public:
    int maximumScore(vector<int>& nums, int k) {

        const long long MOD=1e9+7;
        int n=nums.size();
        vector<int> score(n,0);
        for(int i=0;i<n;i++){

            int temp=nums[i];
            for(int p=2;p*p<=temp;p++){
                if(temp%p==0){
                    score[i]++;
                    while(temp%p==0){
                        temp/=p;
                    }
                }
            }

            if(temp>1){
                score[i]++;
            }
        }

        vector<long long> ways(n);
        for(int i=0;i<n;i++){

            int l=i;
            while(l>=1 && score[l-1]<score[i]){
                l--;
            }

            int r=i;
            while(r<n-1 && score[r+1]<=score[i]){
                r++;
            }

            ways[i]=1LL*(i-l+1)*(r-i+1);
        }

        vector<pair<int,int>> arr;
        for(int i=0;i<n;i++){
            arr.push_back({nums[i],i});
        }
    }
};
```

Code Iteration 2 - Part 2

```
sort(arr.begin(),arr.end(),greater<pair<int,int>>());

long long ans=1;
for(auto it:arr){

    if(k==0) break;
    int val=it.first;
    int idx=it.second;

    long long take=min(ways[idx],(long long)k);
    long long curr=1;

    // while(take>0){
    //     curr=curr*val%MOD;
    //     take--;
    // }

    //this part was leading to a tle

    //so we use binary exponentiation
    //aaj ka gyaan
    //mujhe bhi aaj hi pata chala

    long long base=val;
    while(take>0){
        if(take%2==1){
            curr=curr*base%MOD;
        }

        base=base*base%MOD;
        take/=2;
    }

    ans=ans*curr%MOD;
    k-=min(ways[idx],(long long)k);
}

return ans;
}
```

```
};
```

APPROACH 3: OPTIMAL $O(N)$ USING MONOTONIC STACK

"Pichle waala TLE de rha tha because of the while loops.
So to optimize I'll use monotonic stack. $O(N^2) \rightarrow O(N)$ ho jaayega."

Code Iteration 3 - Part 1

```
class Solution {
public:
    int maximumScore(vector<int>& nums, int k) {

        //pichle waala tle de rha tha because of
        // for(int i=0;i<n;i++){

        //     int l=i;
        //     while(l>=1 && score[l-1]<score[i]){
        //         l--;
        //     }

        //     int r=i;
        //     while(r<n-1 && score[r+1]<=score[i]){
        //         r++;
        //     }

        //     ways[i]=1LL*(i-l+1)*(r-i+1);

        // }

        //so to optimize ill use monotonic stack
        //o(n2)->o(n) ho jaayega

        const long long MOD=1e9+7;
        int n=nums.size();
        vector<int> score(n,0);
        for(int i=0;i<n;i++){

            int temp=nums[i];
            for(int p=2;p*p<=temp;p++){
                if(temp%p==0){
                    score[i]++;
                    while(temp%p==0){
                        temp/=p;
                    }
                }
            }

            if(temp>1){
                score[i]++;
            }
        }
    }
};
```

Code Iteration 3 - Part 2

```
vector<long long> ways(n);
stack<int> st;
for(int i=0;i<n;i++){

    //left me nearest index chahiye jaha
    //score>= ho jaaye
    //tabhi invalid hoga

    ///if top has smaller score it cant stop us
    while(!st.empty() && score[st.top()]<score[i]){
        st.pop();
    }

    //if none can stop us
    if(st.empty()){
        ways[i]=i+1;
        //can start anywhere from 0 to i;
    }

    else{
        ways[i]=i-st.top();
    }

    st.push(i);
}

//now end pos ka logic
//refresh stack

while(!st.empty()){
    st.pop();
}

for(int i=n-1;i>=0;i--){

    //right me nearest index chahiye jaha
    //score> ho jaaye
    //tabhi invalid hoga

    ///if top has smaller or equal score it cant stop us
    while(!st.empty() && score[st.top()]<=score[i]){
        st.pop();
    }

    //if none can stop us
    if(st.empty()){
        ways[i]*=n-i;
        //can end anywhere from n-1 to i;
    }

    else{
        ways[i]*=st.top()-i;
    }

    st.push(i);
}
```

Code Iteration 3 - Part 3

```
vector<pair<int,int>> arr;
for(int i=0;i<n;i++){
    arr.push_back({nums[i],i});
}

sort(arr.begin(),arr.end(),greater<pair<int,int>>());

long long ans=1;
for(auto it:arr){

    if(k==0) break;
    int val=it.first;
    int idx=it.second;

    long long take=min(ways[idx],(long long)k);
    long long curr=1;
    long long base=val;

    while(take>0){

        if(take%2==1){
            curr=curr*base%MOD;
        }

        base=base*base%MOD;
        take/=2;
    }

    ans=ans*curr%MOD;
    k-=min(ways[idx],(long long)k);
}

return ans;
}
};
```